

# Asistente conversacional

Propuesta técnica y de producto · Plataforma AME · Rev. 2 · 04/08/2026

Este documento describe la incorporación de un asistente conversacional a la app mobile de clientas: qué problema resuelve, cómo se construiría sobre lo que la plataforma ya tiene, qué límites debe respetar y en qué momento del plan conviene encararlo. La conclusión corta: **es viable y el proyecto es un sustrato inusualmente bueno**, porque la capa más costosa de este tipo de sistemas —el acceso a datos reales y seguros— ya está construida y probada.

**Sobre esta revisión:** los supuestos de la primera versión se verificaron contra el código de la plataforma. Donde algo ya está resuelto se cita el archivo. Se corrigen dos cosas que la primera versión daba por ciertas y no lo eran —el contenido de las fichas y el alcance sobre productos— y se agrega el único punto que sí implica infraestructura nueva: el impacto del envío incremental sobre el servidor actual (sección 4.1).

## Qué resuelve

El centro responde las mismas preguntas una y otra vez: qué es cada tratamiento, si duele, cuánto dura, qué producto sirve para determinado tipo de piel, cuándo es el próximo turno. Hoy eso ocupa tiempo del staff por canales dispersos. Un asistente dentro de la app absorbe ese volumen y, más importante, **convierte**: puede llevar de una consulta a una reserva o a una recompra sin fricción intermedia.

**Alineación con el negocio:** los dos problemas declarados del centro son la adquisición de clientas y la baja conversión a servicios, con la venta de productos como fuente principal de ingresos. El asistente ataca la conversión a servicios de forma directa y sostiene la venta de productos con la herramienta de catálogo de la sección 1. La adquisición queda fuera de su alcance: son clientas que todavía no tienen la app.

## 1. Lo que ya está construido

La diferencia entre este proyecto y un desarrollo desde cero es que **los datos no son documentos: son registros vivos** —servicios, productos, turnos, la rutina de cada clienta, la disponibilidad—. Eso no se resuelve con búsqueda de texto sino con acceso directo y controlado, y esa capa ya existe: los endpoints de la API de clientes se traducen casi uno a uno en las herramientas del asistente.

| Herramienta del asistente | Endpoint existente                  | Ejemplo de consulta                             |
|---------------------------|-------------------------------------|-------------------------------------------------|
| Buscar tratamientos       | /api/client/turnos/servicios/       | "¿Qué tratamientos hay para el rostro?"         |
| Buscar productos          | /api/public/centros/<id>/productos/ | "¿Qué tienen de la línea facial y cuánto sale?" |
| Mis turnos                | /api/client/turnos/                 | "¿Cuándo tengo turno?"                          |
| Ver disponibilidad        | /api/client/turnos/disponibilidad/  | "¿Hay lugar el martes a la tarde?"              |
| Reservar turno            | /api/client/turnos/ (POST)          | "Sacame turno para el jueves"                   |
| Mi rutina                 | /api/client/mi-rutina/              | "¿Qué me toca de noche?"                        |

Cada una de esas herramientas ya tiene resuelto lo difícil, y no como convención sino como código. El alcance por vinculación lo impone **ClienteScopeMixin** en **apps/client\_api/views.py**, que resuelve la ficha siempre desde la sesión autenticada y nunca desde un identificador del request. La prevención de doble reserva vive en **reservar\_turno()**; la política de días, en el campo **dias\_reserva** de cada servicio; la anticipación mínima, en

**DIAS\_MINIMOS\_ANTICIPACION.** El asistente no reimplementa nada de eso: lo consume.

El tope de uso por clienta tampoco es una idea nueva: la reserva desde la app ya está limitada a veinte por hora mediante el scope `cliente_reserva` en la configuración. El asistente extiende un patrón que ya corre en producción, no inventa uno.

## 2. Arquitectura

La propuesta se organiza en cuatro capas. No todas pesan igual, y ordenarlas mal es el error más común en este tipo de proyectos.

### 2.1 Modelo: instrucciones y uso de herramientas

Es el 80% del valor y por donde se empieza. El asistente recibe un conjunto acotado de herramientas y decide cuál usar según lo que se le pregunta. El ciclo de consulta, llamada a herramienta y respuesta lo maneja el SDK oficial; el trabajo propio está en diseñar bien las herramientas y las instrucciones.

**Invariante de seguridad:** el asistente corre en el servidor con la identidad de la clienta autenticada. Las herramientas resuelven a qué ficha acceden a partir de esa sesión, nunca de un identificador propuesto por el modelo. Es la misma regla que ya rige toda la API de clientes.

### 2.2 Recuperación de contenido

Aplica solo al contenido en prosa: fichas de tratamiento, descripciones de producto, preguntas frecuentes, cuidados. **El modelo de datos para ese contenido ya existe:** el servicio tiene campos de descripción, beneficios y video, y el detalle se sirve por la API pública. Lo que falta es la carga editorial, no el esquema —una diferencia que acorta el prerrequisito, porque es trabajo de contenido y no de desarrollo—. Cuando ese contenido exista, tampoco implica necesariamente infraestructura nueva. La progresión correcta es incremental:

| Escalón                               | Cuándo                                                          | Costo de infraestructura       |
|---------------------------------------|-----------------------------------------------------------------|--------------------------------|
| Todo el contenido en el contexto      | Corpus chico (decenas de páginas). Punto de partida.            | Ninguno                        |
| Herramienta que trae la ficha puntual | El modelo pide un tratamiento, se le da solo esa ficha.         | Ninguno                        |
| Búsqueda de texto de PostgreSQL       | Preguntas frecuentes, búsqueda por palabra clave.               | Ninguno (ya instalado)         |
| Búsqueda semántica con pgvector       | Cuando fallan las paráfrasis ("manchas" / "hiperpigmentación"). | Extensión sobre la base actual |
| Base vectorial dedicada               | A la escala de un centro, prácticamente nunca.                  | Servicio nuevo                 |

Conviene medir antes de decidir: con una ventana de contexto de un millón de tokens y caché de instrucciones, el catálogo completo de un centro entra sin problema y se lee a una fracción del costo. El salto al escalón siguiente se justifica por calidad de respuesta —demasiado contenido irrelevante degrada la precisión— antes que por capacidad.

## 2.3 Ciclo de conversación

Un único ciclo acotado con estado alcanza. El caso que sí requiere varios pasos encadenados es la reserva conversacional: "quiero turno para limpieza la semana que viene a la tarde" obliga a consultar disponibilidad, proponer opciones y confirmar. Son tres o cuatro vueltas al modelo, y eso tiene consecuencias de costo que se detallan en 4.2. Arquitecturas de múltiples agentes coordinados serían sobredimensionadas para este alcance.

## 2.4 Evaluación

Es la capa que decide si el sistema es publicable, y la que más se subestima. Consiste en un conjunto de casos de prueba con respuesta esperada, ejecutado como parte de la suite de tests, más un evaluador automático para las respuestas abiertas. Cubre tres cosas distintas:

- Que responda bien lo que tiene que responder: precios, horarios, disponibilidad, datos propios de la clienta.
- Que se niegue cuando tiene que negarse: toda consulta clínica (ver 3.1).
- Que no obedezca instrucciones que vengan de los datos y no de la clienta (ver 3.4).

El conjunto de casos no hay que inventarlo: el historial de consultas del centro por WhatsApp e Instagram contiene las preguntas reales que hace la gente. Recolectar unas cincuenta de ahí es el primer paso, cuesta nada, y de paso indica si el asistente hace falta o si esas dudas se resuelven con una buena ficha de tratamiento.

## 2.5 Observabilidad y costo

Cada interacción se registra con tokens de entrada y salida, latencia, herramientas invocadas y costo calculado. Con eso se responde "¿cuánto sale?" y "¿qué se está preguntando?" con datos, no con impresiones. Se complementa con un tope de uso diario por clienta para acotar el gasto ante un uso anómalo.

# 3. Restricciones

CRÍTICO

## 3.1 El asistente no da indicaciones clínicas

Es la decisión de diseño más importante del proyecto. Las clientas van a preguntar si pueden hacerse un tratamiento estando embarazadas, si les sirve teniendo rosácea, o si un producto se mezcla con el ácido que ya usan. **Un sistema automático respondiendo eso con el nombre del centro encima es un riesgo real**, no teórico.

La regla es explícita: ante cualquier consulta clínica, el asistente deriva a la profesional. No se resuelve mejorando la información disponible —aunque la ficha tenga las contraindicaciones cargadas, la respuesta correcta sigue siendo derivar—. Es una política, y por eso se verifica con casos de prueba específicos que fallan el deploy si el comportamiento se rompe.

## 3.2 Aislamiento de datos

La plataforma es multi-inquilino: cada centro ve solo lo suyo. El asistente es una superficie nueva sobre esos mismos datos y debe respetar la misma frontera, con el agravante de que es más difícil de auditar a simple vista. De ahí que el alcance se resuelva siempre en el servidor y nunca a partir de lo que el modelo proponga.

Hay un caso que la interfaz resuelve y la conversación no: las cuentas vinculadas a más de un centro. En la app el centro se elige con un selector y viaja como parámetro; en una conversación no hay selector, y sin ese parámetro el backend devuelve la primera vinculación. "Sacame turno el jueves" reservaría en un centro arbitrario. **El centro tiene que quedar fijado al abrir la conversación y viajar en cada llamada a herramienta**, con el mismo rigor que la identidad.

### 3.3 Contenido como prerequisite

El asistente no puede responder mejor de lo que sabe. Los campos ya están en la base; lo que falta es escribirlos. Sin fichas de tratamiento ni descripciones de producto cargadas, el asistente queda limitado a nombre, duración, precio y a los datos propios de la clienta. Por eso el trabajo de contenido —que ya está planificado— es condición previa, no una mejora posterior.

### 3.4 Lo que el asistente lee no es confiable

La sección 3.2 cubre que el modelo no elija a qué ficha acceder. Falta la mitad simétrica: el texto que las herramientas devuelven entra al contexto del asistente y puede contener instrucciones. No es hipotético. El alta de turno desde la app acepta un campo de notas escrito por la clienta, y ese texto se devuelve después al listar los turnos: alcanza con escribir ahí lo que se quiera para que el asistente lo lea en la conversación siguiente. Lo mismo vale para cualquier campo que cargue el staff.

La regla es tratar todo lo que sale de la base como dato y nunca como instrucción: contenido de terceros delimitado y etiquetado como tal dentro del mensaje, la política de derivación clínica por encima de cualquier cosa que devuelva una herramienta, y casos de prueba adversarios con instrucciones incrustadas en notas y descripciones.

## 4. Decisiones técnicas

El backend es Python con Django, así que la integración usa el SDK oficial de Anthropic para Python. Las decisiones relevantes, y sobre todo lo que se decide no usar:

| Decisión               | Elegido                               | Motivo                                                                                                                                                                                                                                                                                       |
|------------------------|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Acceso al modelo       | SDK oficial, invocación directa       | El ciclo de herramientas viene resuelto por el SDK. Un framework intermedio agrega abstracción sobre pocas líneas de código y oculta lo que conviene entender.                                                                                                                               |
| Modelo                 | Uno solo para empezar                 | Rutear por tipo de consulta suena barato, pero exige clasificar antes de responder: o es otra llamada al modelo —la latencia y el costo que la ruta económica quería evitar— o es una heurística frágil. Se arranca con un modelo y se rutea cuando el registro de uso muestre que conviene. |
| Respuesta en pantalla  | Envío incremental                     | La respuesta aparece a medida que se genera. Sin esto la espera se percibe como que la app se colgó. Tiene una consecuencia sobre el servidor: ver 4.1.                                                                                                                                      |
| Servidor del chat      | Proceso aparte con workers asíncronos | El envío incremental no convive con el servidor sincrónico actual. Es el único punto de la propuesta que implica infraestructura nueva.                                                                                                                                                      |
| Caché de instrucciones | Activa desde el inicio                | El texto fijo y el catálogo se cachean y se releen a una fracción del costo. Es la palanca de ahorro más grande, y la que hace viable el ciclo de varias vueltas.                                                                                                                            |
| Evaluación             | Suite propia sobre pytest             | A esta escala, un conjunto de casos propio es más simple y más transparente que adoptar una herramienta externa.                                                                                                                                                                             |

| Decisión       | Elegido       | Motivo                                                                                            |
|----------------|---------------|---------------------------------------------------------------------------------------------------|
| Base vectorial | No, por ahora | Ver la progresión de la sección 2.2. Cuando haga falta, es una extensión de la base que ya corre. |

#### 4.1 Impacto sobre la infraestructura actual

Es el único punto de esta propuesta que sí implica trabajo de infraestructura, y conviene decirlo con precisión porque matiza el "no hace falta nada nuevo" del resto del documento.

El backend se sirve hoy con Unicorn sobre WSGI y tres workers sincrónicos. Un worker sincrónico atiende una request por vez, de principio a fin. Una respuesta en streaming mantiene la conexión abierta durante toda la generación —varios segundos—, así que ocupa un worker todo ese tiempo. **Tres conversaciones simultáneas dejan al backend sin workers**, y lo que se cae no es el chat: es la API entera, incluidos turnos, dashboard y administración.

Tres salidas, de menor a mayor alcance:

- **Workers asíncronos en el proceso actual.** Cambio de una línea en el arranque, pero obliga a revisar que todo lo bloqueante del backend sea compatible.
- **Servir el chat como proceso aparte sobre ASGI.** Aísla el problema y permite dimensionar esa ruta por separado; agrega un servicio al despliegue.
- **Migrar el backend entero a ASGI.** Es lo correcto a largo plazo y lo más caro ahora.

La recomendación es la segunda: el chat es la única ruta con este perfil de conexión larga, aislarla no arrastra al resto de la API y deja la migración completa para cuando haya un motivo propio. **La decisión hay que tomarla antes de escribir el endpoint**, no después.

#### 4.2 Orden de magnitud del costo

Estimación con supuestos explícitos: instrucciones y catálogo cacheados en torno a quince mil tokens y una respuesta final de unos cientos de tokens. El supuesto que falta en la cuenta ingenua es el ciclo: una reserva conversacional son tres o cuatro llamadas al modelo, y cada una relee el contexto completo más lo que devolvieron las herramientas anteriores. La caché hace que esa relectura salga una fracción, pero hay que contarla: multiplica.

- Con un modelo económico: del orden de medio centavo de dólar para una consulta simple de catálogo, y unos pocos centavos para una conversación con reserva.
- Con un modelo de alta capacidad: unos pocos centavos la consulta simple, y hasta un orden de magnitud más en una conversación con varias herramientas encadenadas.

A volúmenes de un centro de estética el costo del modelo sigue sin ser el factor limitante. Aun así se instrumenta desde el principio: la cifra hay que medirla, no suponerla, y el tope diario por clienta acota cualquier uso anómalo.

### 5. Alcance de la primera versión

Deliberadamente angosta. El objetivo de la primera versión no es cubrir todo, sino validar que las clientas efectivamente la usan y que el comportamiento es seguro.

## Incluye

- Consultas sobre el catálogo de tratamientos: qué hay, duración, precio, en qué días se pueden reservar.
- Consultas sobre el catálogo de productos: qué hay, de qué línea y a qué precio.
- Consultas sobre datos propios: próximos turnos, historial, rutina de cuidado.
- Reserva de turno asistida, con confirmación explícita antes de crear nada y con el centro fijado desde el inicio de la conversación.
- Negativa y derivación ante cualquier consulta clínica.
- Registro completo de cada interacción con su costo, y tope diario por clienta.
- Conjunto de casos de evaluación construido a partir de consultas reales del centro, con casos adversarios clínicos y de inyección.

## No incluye

- Búsqueda semántica sobre contenido en prosa (llega cuando el contenido exista y lo justifique).
- **Stock o disponibilidad de producto.** El inventario de la plataforma todavía no refleja el real: el asistente informa qué hay en catálogo y a qué precio, nunca cuánto queda.
- Cancelación o reprogramación por el asistente: son acciones destructivas y quedan en la interfaz, donde la confirmación es inequívoca.
- Recomendación de tratamientos o productos según características de la piel: es consejo clínico.
- Cualquier operación de pago.

## Cómo se mide si funcionó

Tres indicadores, definidos antes de construir:

- **Uso:** qué proporción de clientas activas lo abre, y cuántas vuelven a usarlo.
- **Conversión:** cuántas reservas se originan en una conversación con el asistente.
- **Seguridad:** proporción de consultas clínicas correctamente derivadas, medida sobre el conjunto de casos y sobre conversaciones reales revisadas a mano.

## 6. Riesgos

| Riesgo                                                      | Mitigación                                                                                                                                                                              |
|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| El asistente responde una consulta clínica                  | Instrucción explícita de derivación, casos de prueba adversarios que frenan el deploy, y revisión manual de conversaciones durante las primeras semanas.                                |
| El streaming satura los workers y cae toda la API           | Aislar la ruta del chat en un proceso con workers asíncronos antes de exponerla, y probarla con conexiones concurrentes. Ver 4.1.                                                       |
| Instrucciones incrustadas en los datos que el asistente lee | Contenido de terceros delimitado y etiquetado dentro del mensaje, política de derivación por encima de lo que devuelva cualquier herramienta, y casos adversarios en la suite. Ver 3.4. |

| Riesgo                                                 | Mitigación                                                                                                                         |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Afirma datos incorrectos (precios, horarios)           | Los datos salen siempre de herramientas contra la base, nunca de la memoria del modelo. Lo que no puede consultar, no lo responde. |
| Alguien intenta acceder a datos de otra clienta        | El alcance se resuelve en el servidor a partir de la sesión autenticada. El modelo no puede proponer a qué ficha acceder.          |
| Reserva en el centro equivocado (cuentas multi-centro) | El centro queda fijado al abrir la conversación y viaja en cada llamada a herramienta. Ver 3.2.                                    |
| Costo mayor al esperado                                | Caché de instrucciones, registro por interacción con el ciclo completo contabilizado, y tope diario por clienta.                   |
| Nadie lo usa                                           | Es el riesgo más probable y por eso la primera versión es chica. Se mide uso antes de invertir en las capas siguientes.            |

## 7. Ubicación en el plan

El asistente se apoya en trabajo que ya está planificado y que hay que hacer de todos modos. Esta es su posición respecto del resto:

| Antes                                  | Por qué es prerequisite                                                                                                                                                                                                       |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Contenido de las fichas de tratamiento | Los campos ya existen en la base; falta escribirlos. Sin descripción, beneficios y cuidados cargados, el asistente no tiene de dónde responder más allá de nombre, duración y precio. Es trabajo editorial, no de desarrollo. |
| Descripciones de producto              | Mismo motivo, del lado del catálogo de productos, que es la fuente principal de ingresos del centro.                                                                                                                          |
| Almacenamiento de imágenes en la nube  | Necesario para que las fichas tengan foto y sobrevivan a un despliegue.                                                                                                                                                       |
| Decisión sobre el servidor del chat    | La opción de 4.1 condiciona el despliegue. Conviene resolverla antes de escribir el endpoint, no después.                                                                                                                     |
| Notificaciones push                    | No es estrictamente necesario. El registro de dispositivo ya está resuelto; falta el envío. Comparte la infraestructura de comunicación con la clienta y conviene tenerlo cerrado antes.                                      |

### 7.1 Orden de magnitud del esfuerzo

Con los prerequisites cubiertos, y para el alcance de la sección 5:

- Herramientas sobre los endpoints existentes, instrucciones y ciclo de conversación: una semana.
- Aislamiento del proceso de chat y envío incremental: entre media semana y una y media, según la salida de 4.1 que se elija.
- Suite de evaluación con casos clínicos y de inyección: una semana. No es opcional: es lo que decide si se publica.
- Registro de uso, costo y tope diario: media semana.

- Interfaz de chat en la app: una semana.

Del orden de cuatro a cinco semanas de trabajo enfocado. Es una estimación, no una cotización: el número que más se mueve es el de la evaluación, y se mueve según qué tan sucio venga el conjunto de consultas reales.

Con esos puntos cubiertos, el asistente pasa de ser una funcionalidad aislada a ser la capa que conecta lo que ya existe: catálogo, turnos, rutina y recompra, en una sola conversación. Ese es el argumento para ubicarlo al final y no antes: no es un desvío del plan, es lo que lo corona.

### **Primer paso, que no requiere escribir código**

Recolectar unas cincuenta consultas reales del historial de WhatsApp e Instagram del centro. Sirve para tres cosas a la vez: es el conjunto de evaluación, indica qué herramientas hacen falta de verdad, y revela si esas dudas se resuelven con una buena ficha de tratamiento en lugar de un asistente. Es la forma más barata de decidir si este proyecto vale la pena antes de invertir en él.